



Práctica 1: Análisis de algoritmos

Algorítmica

Víctor M. Campello

Curso 2025/2026

Descripción

En esta práctica analizaréis un conjunto de algoritmos mencionados en clase. Para ello, se os facilitan las implementaciones en C++ de los algoritmos y se os pide diseñar el código que ejecutará cada algoritmo para diferentes tamaños de n y guardará su tiempo de ejecución.

Una vez recopilados los tiempos de ejecución para los diferentes algoritmos y para diferentes valores de n , deberéis comparar el tiempo de ejecución con la predicción teórica dada por su orden exacto, Θ .

Como librerías, necesitaremos `iostream` –para entrada y salida de texto por terminal–, `cstdlib` –para generar números aleatorios–, `ctime` –para calcular el tiempo de ejecución–, y `fstream` –para guardar en fichero–.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <fstream>
```

```
using namespace std;
```

Pasaremos el algoritmo a ejecutar y el tamaño de la instancia por línea de comandos, por lo que usaremos la función `main` con los parámetros siguientes:

```
int main(int argc, char *argv[]) {
    // Cuerpo de main
```

```

    return 0;
}

```

Las primeras líneas del código deben declarar las variables necesarias para poder representar el vector a ordenar (en nuestro caso, supongamos que son números enteros), los tiempos antes y después de la ejecución del algoritmo, y el fichero de salida donde se guardarán los resultados:

```

int *v; // Puntero para crear el vector de valores
int n, i, argumento;
unsigned long int tini, tfinal; // Tiempos de ejecución inicial y final
double tejecucion; // tiempo de ejecución del algoritmo en ms
unsigned long int semilla; // Semilla para el generador de números aleatorios
ofstream fsalida; // Fichero de salida

```

El programa tiene un número de parámetros de entrada variable. El primer parámetro (después del propio nombre del programa) debe ser el nombre del algoritmo a ejecutar. Los parámetros restantes se corresponden con la semilla para el generador de números aleatorios y diferentes tamaños de casos que se deberán probar. Por tanto, las siguientes líneas del código deben comprobar que el número de parámetros del algoritmo es correcto, que el algoritmo seleccionado existe y que el fichero de salida se abre correctamente. Por último, se inicializa el generador de números aleatorios:

```

if (argc <= 3) {
    cerr << "\nError: El programa se debe ejecutar ";
    cerr << "de la siguiente forma.\n\n";
    cerr << argv[0];
    cerr << " NombreAlgoritmo Semilla tamCaso1 tamCaso2 ... tamCasoN\n\n";
    return 0;
}

// Abrimos fichero de salida con el mismo nombre que el algoritmo
fsalida.open(string(argv[1])+".dat");
if (!fsalida.is_open()) {
    cerr << "Error: No se pudo abrir fichero para escritura "
    cerr << argv[1] << "\n\n";
    return 0;
}

// Inicializamos generador de números aleatorios
semilla = atoi(argv[2]);
srand(semilla);

```

El siguiente fragmento de código debe recorrer todos los tamaños de caso de entrada al programa, generando un vector aleatorio de cada tamaño de caso y ejecutando el algoritmo con dicho vector como entrada. El tiempo actual del computador (en ticks de reloj) se puede

obtener mediante la función `clock`. De este modo, si hacemos las variables `tini=clock()`; y `tfin=clock()`; antes y después de la ejecución del algoritmo, obtendremos los tiempos donde comienza y termina la ejecución del algoritmo. Utilizando la diferencia `tfin-tini`, podemos calcular el tiempo en milisegundos que tarda en ejecutarse el algoritmo y pasarlo a segundos mediante la constante `CLOCKS_PER_SEC`. El resultado deberá ser guardado a fichero.

```
// Pasamos por cada tamaño de caso
for (int argumento=3; argumento<argc; argumento++) {
    // Cogemos el tamaño del caso
    n = atoi(argv[argumento]);

    // Reservamos memoria para el vector
    v = new int[n];

    // Generamos vector aleatorio de prueba, con componentes entre 0 y n-1
    for (i= 0; i<n; i++)
        v[i]= rand()%n;

    tini = clock(); // Tiempo en que comienza la ejecución
    // Ejecutamos algoritmo para tamaño de caso n
    AlgoritmoOrdenamiento(v, n);
    tfin = clock(); // Tiempo en que finaliza la ejecución

    tejecucion= 1000*(tfin-tini)/(double)CLOCKS_PER_SEC;
    // Guardamos tam. de caso y t_ejecucion a fichero de salida
    fsalida << n << " " << tejecucion << "\n";
    // Lo mostramos tambien por pantalla
    cerr << "Tam. Caso: " << n << " T. Ejecucion: " << tejecucion << endl;

    // Liberamos memoria del vector
    delete [] v;
}
```

Por último, al finalizar el bucle anterior, debemos cerrar el fichero de salida de datos y salir del programa:

```
fsalida.close();
return 0;
```

Cálculo de la eficiencia empírica

Para llevar a cabo un estudio puramente empírico del comportamiento de los algoritmos analizados, mediremos los recursos empleados (tiempo, en este caso) para cada tamaño dado de las entradas. En el caso de los algoritmos de ordenación, el tamaño viene dado por el número de componentes del vector a ordenar. En otro tipo de problemas, como es el caso

Table 1: Eficiencia teórica de los algoritmos considerados.

Algoritmo	Eficiencia
bubble	$O(n^2)$
insertion	$O(n^2)$
selection	$O(n^2)$
mergesort	$O(n \log n)$
quicksort	$O(n \log n)$
heapsort	$O(n \log n)$
fibonacci	$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$
hanoi	$O(2^n)$

del algoritmo para resolver el problema de las torres de Hanoi, el tamaño se corresponde con el valor del entero que representa el número de discos.

Una vez tengamos los resultados guardados en un fichero, queremos comparar la eficiencia empírica con la eficiencia teórica. Para ello, hacemos uso de la Tabla 1 para calcular el ratio entre el valor empírico y el valor teórico:

$$K = \frac{T(n)}{f(n)}. \quad (1)$$

Este ratio debería tender a un valor constante, el parámetro mediante el cual acotamos el tiempo de ejecución. Es decir, un valor $\hat{K}$ tal que $T(n) \leq \hat{K}f(n)$, $\forall n \geq n_0$.

Tareas de la práctica

Ejercicio 1. (2.5 puntos) Cread el programa para ejecutar los diferentes algoritmos mediante parámetros por línea de comandos. Por ejemplo, el comando

```
./main insertion 1234 1000 2500 5000 10000
```

deberá establecer la semilla 1234 y ejecutar el algoritmo de ordenación `insertion` con arrays aleatorios de tamaños 1000, 2500, 5000 y 10000 y guardar los resultados en un fichero con nombre `insertion.dat`.

Ejercicio 2. (2.5 puntos) Calculad el tiempo de ejecución de cada uno de los algoritmos proporcionados y guardadlo a fichero para 25 tamaños diferentes de casos de cada problema. El tamaño de caso de cada alumno será diferente y dependerá de la potencia de cálculo de su PC. Evitad tamaños que requieran un excesivo tiempo de ejecución.

Ejercicio 3. (2.5 puntos) Calculad el valor de las constantes ocultas, $\hat{K}$, de cada uno de los algoritmos.

Ejercicio 4. (2.5 puntos) Mostrad la curva ajustada de los valores empíricos y la función $\hat{K}f(x)$ para cada algoritmo. Podéis usar la herramienta `gnuplot` o cualquier otra de vuestro gusto.

Todos los resultados se presentarán en una memoria sencilla pero bien escrita que se deberá entregar en formato PDF y debidamente formateada.